

Implementation of a Real Time Audio Ray Tracing & Occlusion system using Unity & Csound

Bilkent Samsurya, Undergraduate, Berklee College Of Music, Dr Richard B, Berklee College Of Music

Abstract: Interest in interactive virtual environment design has increased dramatically over the past decade. While there has been much research done in visual rendering, studies on audio technology are lacking. Yet, there exist many acoustical modeling tools which can be applied to audio processing for virtual environments. This thesis describes an implementation of a real-time ray tracing and sound occlusion system to model acoustics in a virtual environment. The game engine Unity is chosen for the environment and Csound is the sound system chosen for the audio processing component. The first part of the study reviews acoustical concepts that are considered in the conception of this program. It then gives a definition of audio ray tracing and sound occlusion before explaining their significance to simulating a compelling acoustical environment. The scope and capabilities of the system are outlined, detailing the acoustical properties it accounts for. An overview of the rays generation, detection process programmed in Unity and the equations, components used in Csound to generate the reverberations and occlusion effect is discussed. Four enclosures of varying sizes, shapes and materials are constructed in Unity to test the effectiveness and efficiency of the system. The frequency response and decay times of the four environments are collected and analyzed. The limitations of the system are outlined and possible future work to explore different reverb algorithms is discussed.

1.1 Introduction

The interest in game development has increased dramatically over the past decade with easily interfaceable game engines like Unity and Unreal Engine gaining more notoriety. The advent of these technologies has garnered more research into interactive virtual environments resulting in the creation of more versatile and powerful tools for game developers to more efficiently design their games with. While there have been great strides made in the visual aspect of game development (i.e Lumen: a global illumination system and light reflections made in Unreal 5, Nanite : a virtualized geometry system made in Unreal 5), the development of its audio counterpart seems to have been overshadowed. To create a more seamless and immersive experience, there needs to be a greater emphasis in the development of audio tools to complement the improvement in the visual aspect of game design [8].

The current state of audio implementation in games is lacking in a few areas. The first is that they do not account for audio reflections and subsequently, reverberations very well. Reverberation is one of the most vital acoustical properties in a room [2]. It can provide auditory information about the size, texture and spatialization to the listener. The conventional method for emulating reverberations in virtual environments is to bake reverb zones within designated spaces in the game environment by utilizing reverb plugins in audio middlewares. The parameters of these plugins have to be tweaked to match the sonic environment in the scene, and this process is repeated for every other area within the game. It can become repetitive especially when there is a large diversity of areas in the game. More importantly, they do not account for spatialization of the reflected sounds. Take for instance, a scenario where the player is walking in a large chamber. The reverb amount applied onto the player's footsteps should change depending on where the player is situated. More specifically, we should hear a slight increase in reverb coming from the right if there is a wall closer to the

right of the player compared to the case when the player is at the center of the space. The current implementation of reverbs does not account for this change. The second drawback is that sound impedance is not accounted for in the acoustical environment. Sound Impedance occurs when a significantly large object obstructs the listener and sound source. Lower frequencies with longer wavelengths are able to travel around the obstacle and reach the listener while higher frequencies with smaller wavelengths do not [2]. While sound impedance can be emulated by writing a custom script to work with certain audio middleware, this method offers limited customizability and requires the user to write their own scripts. The inclusion of these sound behaviors in game design can go a long way as it can better help players navigate the environment, enhancing the overall experience by making it more immersive and believable.

1.2 Scope, Functionality & Features

To build a system that can simulate all known sound phenomena to within a 100 % accuracy would be extremely complex if not downright impossible [4]. The aim of this project is not to create an acoustics simulator but to build upon the functionalities of existing game audio implementation softwares by designing a system that repurposes acoustical modeling methods to cater to game sound implementation so as to more broadly and accurately capture sound characteristics in virtual environments. The requirements for design of the program were as follows:

1. Simulate audio reflections that can respond according to the size and material(s) of the enclosure with respect to the player's position within the space. This would mean that the program has to be able to:
 - a. attenuate volume of reflections based on distance of path taken from source to receiver
 - b. attenuate frequency response of reflections based on absorption coefficients of materials of objects encountered by the reflected sound.

- c. determine delay & decay times of reflected sounds based on materials encountered and distance traveled from the source to listener.
 - d. spatialize reflected sounds so as to provide more audio cues about the environment from an acoustical standpoint
2. Simulate sound impedance when there is an obstacle between the listener and sound source. For this to occur, the program has to
 - a. Detect the presence of obstruction between source and listener
 - b. Attenuate frequency response of the direct sound based on the material and amount of obstruction between the source and listener

An underlying theme when programming these functionalities is to ensure that each parameter and variable is easily customizable when appropriate and have an intuitive interface that sound designers of all levels can use.

2.1 Previous Work

There has been significant work done in modeling sound propagation for architectural applications. There is potential for many of these techniques to be applied to the improvement of audio implementation tools for games. Briefly, prior methods can be classified into two major types: 1) numerical solutions to the wave equation using Finite/Boundary Element Methods (FEM/BEM) and 2) high frequency approximations based on geometrical propagation paths. In the latter case, image source methods, ray tracing, and beam tracing have been used to construct the sound propagation paths [9]. I opted for the ray tracing method because it can be generalized to model sound propagations in any geometric shapes. This is important for game design purposes as it does not set constraints on design choices.

2.2 Background : Acoustics Theory

Sound is a wave phenomenon but modeling it as a wave motion requires the solving for complex integrals of wave fields equation [3]. These equations can be complicated to solve and other more intuitive models can be employed in its place. An alternative is to conceptualize sound propagation as discrete rays similar to modeling light rays for global illumination [9].

Assuming the model of using rays as the framework for modeling propagation of sound, there are a number of variables to consider.

The first is frequency range. The audible range of human hearing resides in the 20 Hz - 20KHz range. This range of sound frequencies can be broken down into $\frac{1}{3}$ octave bands, with a lower, center and upper frequency value which can be useful for performing digital signal processes.

The second is the speed of sound. A multitude of factors can affect the speed of sound : temperature, humidity and medium in which the sound is propagating. For the purposes of this study, we will be assuming that sound is traveling through air at a temperature of ___, which results in the speed of sound to be 343 m/s.

The third factor to consider is distanced based volume attenuation. The intensity level I_1 from a distance d_1 can be determined if the sound intensity I_0 at distance d_0 from the source is known. This relationship can be expressed by the following equation:

The fourth phenomenon to consider is reflection. When sound encounters a boundary or obstacle that is significantly larger than the wavelength of the sound frequencies, it gets reflected, absorbed and/or transmitted. [] In the case of reflection, two forms of reflections can occur, specular and diffuse. Specular reflection occurs when the reflected ray bounces off the surface at the same angle as the incident ray. Diffuse reflection occurs when the reflected ray bounces off the surface at a larger angle than the incident ray encountering the surface. When the reflections are diffused, the sound is said to be scattered [8]. Figure 1.1 illustrates the distinctions between these two forms of reflections.

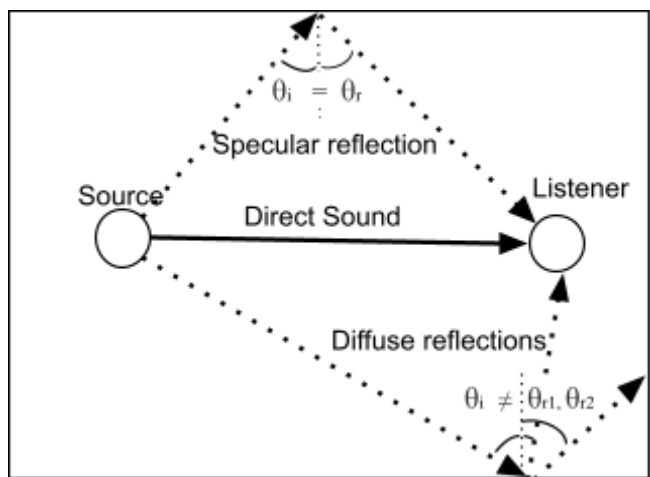


Figure 2.1 : Illustration of the difference between specular & diffuse sound reflections

Another phenomenon is sound absorption which can be classified in 2 categories: 1. Air absorption 2. Wall absorption [3]. Wall absorption occurs when the sound loses energy upon collision with a wall/obstacle that is significantly larger than the wavelength of all frequencies contained within the sound. This results in the uneven attenuation of frequencies of the original sound which varies depending on the acoustical properties of the wall (i.e material). A measure for the wall's absorption capabilities can be obtained from experimentally obtained frequency dependent absorption coefficient data.

The final factor to consider is sound impedance. Sound impedance refers to the attenuation of the sound when it passes through an obstacle to reach the listener. The material, thickness and density of the obstacle determine the amount and quality of sound transmitted loss factor. The measure of sound transmit loss can be obtained from experimental data.

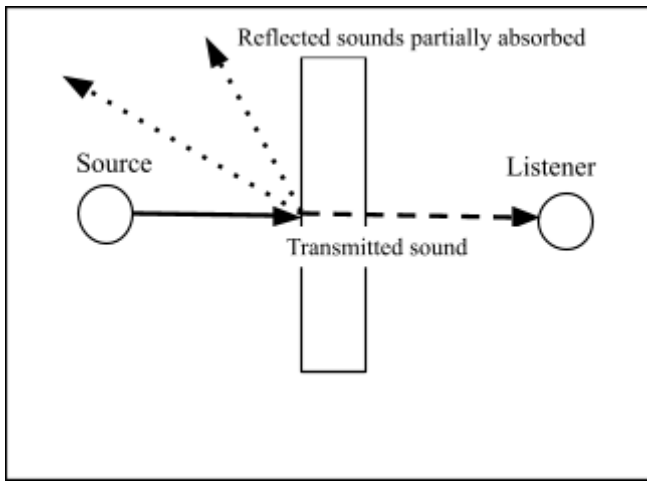


Figure 2.2 : Illustration of the absorption of reflections and transmission loss of direct sound

There are more sound behaviors that are not discussed in this paper. The goal of this project is not to build an acoustics simulator but to repurpose some of their functionalities for game design.

3.1 Program Architecture

This research describes an implementation of a real time 1.audio ray tracing system that models the propagation of specularly reflected sounds from the sound source to the listener to create reverberations and 2. a sound occlusion system that models sound impedance when a significantly large object is obstructing the direct sound path in a virtual environment. The virtual environment is built and run in the Unity game engine. The ray generation, detection process and the data required to perform sound alteration processes is stored and updated in Unity. When the appropriate conditions are met, the data and sound file is sent from Unity to Csound for signal processing (filtering, eqs, delays). Csound alters the sound based on the data sent from Unity and the altered sound is sent back to Unity to be played. This process is done in run time allowing for users to be able to modify that data and hear the evolved audio signal in real time. The implementation of this project can be conceptualized as having two components, one to handle the reflections and the other to account for sound impedance. I will first be going through the sound reflection aspect of the program. A high level depiction of the pipeline is illustrated in figure 3.1.

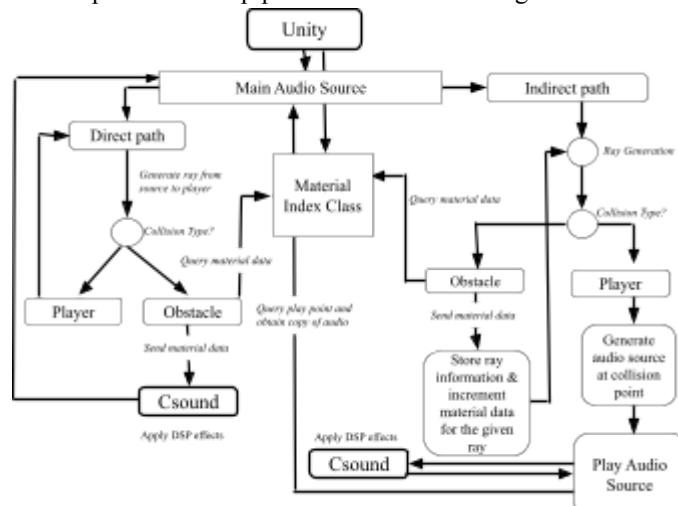


Figure 3.1 : Flowchart of program pipeline

3.2 Ray Generation & Detection

A number of rays (a variable that can be determined by the user) are generated in randomly set directions (sound source polarity assumed to be omnidirectional) from the position of the audio source. If a ray collides with an object that is not the player, the data associated with the material of the object, length of the ray and angle between the normal of the object's surface and the ray is queried and stored. An illustration of the generated rays are illustrated in Figure 3.1.

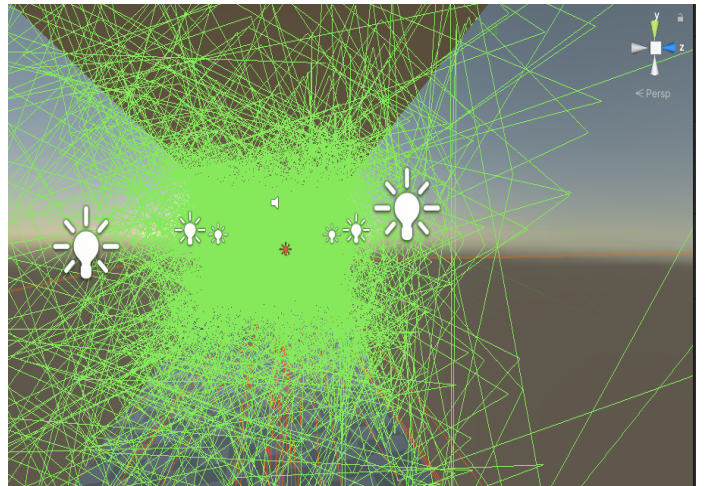


Figure 3.2 : Generated rays to model sound propagation paths in Unity environment

A new ray is generated from the point of collision at an angle from the normal of the surface of the collision. The angle of the reflected ray is equivalent to the angle of the previous ray, ensuring that the reflection is specular ($\theta_i = \theta_r$). The ray will continue to propagate until one of three conditions are met:

1. The number of reflections has been exceeded. The number of reflections represents the order of reflections the system takes into account. This variable, although customizable by the player, should be kept low (4 or below). This is to ensure that Unity is able to run at acceptable frame rates (>30fps) and latency is kept low.
2. The total distance traveled by the sum of the rays has exceeded a maximum threshold. This threshold is to ensure that the sound path does not propagate indefinitely. It is a user-modifiable variable so as to allow users to match the propagation paths to take into account the enclosures of varying sizes and geometries.
3. The ray collides with the player. The method for detecting ray collision in Unity is employed by placing a collider around the player/listener. The most common implementation of this form of detection is to use a spherical collider, to ensure that the collider that there is an equal likelihood of collision from all directions[]. An important consideration is the size of the sphere. This is because the size of the collider will affect the probability of collisions. If the collider is too

large it might result in the oversampling of rays which can result in a false acoustical image of the space. Similarly, a collider that is too small may result in the undersampling of rays which leads to an incomplete acoustical image of the space. It is shown in [] that the minimum radius of the receiver can be determined by the following equation :

$$r = \sqrt{\frac{V}{\pi c \Delta t N}} \quad (3.1)$$

r is the radius of the spherical collider, *V* is the volume of the room, *c* is the speed of sound, Δt is audio sampling period, *N* is the number of rays

When the ray collides with the player, a secondary audio source is instantiated at the collision point, acting as the sound source for each ray of reflected sound. A drawback from this implementation is that the number of secondary audio sources instantiated is equivalent to the number of collisions between the rays and player. This would be a problem if the number of collisions detected is high (i.e 50), as this would mean that 50 audio sources would have to be instantiated. A workaround is to limit the number of audio sources instantiated. To make up for the limited number of secondary audio sources, each audio source would have to be more reverberant and contain higher echo densities, with each ray acting more as a collection of sound paths.

3.3 Frequency & Volume Attenuations

For every collision between a ray and a non-player object, the material of the object is queried and stored. If the ray continues to propagate and eventually collides with the player, the history of all the objects it has collided with is queried and passed into the Material Index Class. The Material Index script is a class which contains acoustical properties of each material found in the game. The absorption coefficients of the materials are obtained from []. The user also has the option of creating a custom material and inputting their own acoustical properties into the class if they so wish. The acoustical properties contained within the class include the sound absorption coefficients of respective materials from frequency bands of 63Hz to 8KHz. The table below[] shows the materials and their corresponding absorption coefficients.

Material	63 Hz	125 Hz	250 Hz	500 Hz	1k Hz	2k Hz	4k Hz	8k Hz
Granite	0.01	0.01	0.01	0.01	0.02	0.02	0.02	0.02

Painted Concr. block	0.10	0.10	0.05	0.06	0.07	0.09	0.08	0.08
Glass Plane	0.18	0.18	0.06	0.04	0.03	0.02	0.02	0.02
Steel Door	0.05	0.05	0.05	0.05	0.06	0.04	0.02	0.02

Figure 3.2 : Absorption Coefficients of Materials []

The absorption coefficients of all encountered materials are incrementally added per frequency band before being passed to Csound for sound processing.

3.3.1 Sound Processing in Csound

The first task of Csound is to break the sound into 9 discrete frequency bands : 63Hz, 125Hz, 250Hz, 500Hz, 1KHz, 2KHz, 4KHz, 8KHz and 16kHz. This is done by applying bandpass filters, using the butterbp opcode. Each of these signals are then attenuated based on the absorption coefficient values associated with the frequency band. The pareq opcode is used for the attenuation. To convert the absorption coefficient to dB, the following equation was used.

$$\Delta dB = 20 \log_{10}(1 - \alpha)$$

a is the absorption coefficient

The absorption coefficients of frequency bandwidths that add up to more than 1 are truncated to 0.99.

3.3.2 Reverb Design using Csound

In designing the reverb for the reflected sound, analysis of previous work done on reverb designs were conducted. Historically, algorithmically generated reverbs were accomplished by duplicating and randomly delaying the signal by a certain amount. This resulted in the reverb sounding artificial as it failed to account for more complex reflections and evolving echo densities. Schroeder's reverb implementation made use of 4 comb filters in parallel and a number of allpass filters in series.

Citations:

- [3] Elorza, D. O. (2005). Room acoustics modeling using the raytracing method: implementation and evaluation. [MAIN]
- [4] A. M. Ondet, J. L. Barbry. Modeling of sound propagation in fitted workshops using ray-tracing. J. Acoust. Soc. Am. 85(2), 787-796, 1989.
- [8] Y. W. Lam. A comparison of three diffuse reflection modeling methods used in room acoustics computer models. J. Acoust. Soc. Am. 100(4) Pt. 1, 1996, 2181-2192.
- [9] Funkhouser, Thomas & Tsingos, Nicolas & Jot, Jean-Marc. (2003). Survey of Methods for Modeling Sound Propagation in Interactive Virtual Environment Systems. [[MAIN]]
- [5] J. H. Rindel. The use of computer modelling in room acoustics. Journal of Vibroengineering, No 3(4), 219-224, 2000.
- [6] J. H. Rindel. The use of computer modelling in room acoustics. Journal of Vibroengineering, No 3(4), 219-224, 2000.
- [7] P. C. Eleftheriou. Industrial noise and its effects in human hearing. Appl.Acoust. 63, 35-42, 2002.

Reverb Studies

- [10]